

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 2

DCL: usuarios, roles y privilegios

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Usuarios y privilegios en MySQL	2
2.1. Conexión al servidor	2
2.2. Creación de usuarios	2
2.3. Otorgamiento de privilegios	3
2.4. Verificación de privilegios	3
2.5. Revocación de privilegios de vendedor2	3
3. Ejercicio 2: Conexión por host y límites de recursos	4
3.1. Creación de usuarios con alcance de host	4
3.2. Otorgamiento de privilegios y límites	4
3.3. Verificación de permisos y de la configuración de cuenta	5
3.4. Revocación del privilegio de consulta del cobrador	5
4. Ejercicio 3: Roles y herencia en PostgreSQL	5
4.1. Pre-chequeo del esquema	5
4.2. Creación del rol común y de los usuarios	6
4.3. Otorgamiento de privilegios	6
4.4. Comprobación de permisos	6
4.5. Resultado esperado por inciso	7
5. Ejercicio 4: Perfiles, vistas y delegación en Oracle	8
5.1. Nota sobre Oracle y privilegios por columna	8
5.2. Bloque de administración (conexión como SYS)	8
5.3. Bloque de privilegios sobre objetos (conexión como PRACTICO1C)	9
5.4. Verificación	9
5.5. Resultado esperado por inciso	10
6. Conclusión	10

1. Introducción

Este práctico aborda DCL, el subconjunto de SQL dedicado al control de acceso. La consigna se divide en cuatro ejercicios y, en conjunto, ejercita las tres operaciones centrales del subsistema de seguridad: crear usuarios y roles, otorgar privilegios y revocarlos. La cátedra exige resolver cada ejercicio en un motor específico (MySQL, PostgreSQL u Oracle), lo que también permite contrastar las diferentes formas en que cada motor implementa la seguridad: a nivel de cuenta y host en MySQL, mediante roles unificados con `LOGIN/NOLOGIN` en PostgreSQL, y con perfiles y vistas para cubrir restricciones particulares en Oracle.

Las bases utilizadas son las definidas en el Práctico 1: la del inciso a) (clientes, productos, facturas) para el Ejercicio 1, la del inciso b) (vehículos, parquímetros, estacionamiento) para el Ejercicio 2 y la del inciso c) (clientes, automóviles, accidentes) para los Ejercicios 3 y 4.

A lo largo del documento se priorizan dos criterios. Primero, otorgar siempre el privilegio mínimo necesario: cuando un inciso pide acceder solo a ciertas columnas, se utiliza la sintaxis a nivel de columna en lugar de un permiso amplio sobre la tabla completa. Segundo, justificar el uso (o la ausencia) de `WITH GRANT OPTION`: solo se incluye en los casos en los que el enunciado pide explícitamente que el usuario pueda delegar el privilegio.

2. Ejercicio 1: Usuarios y privilegios en MySQL

Este ejercicio trabaja sobre la base `practico1a_mysql`, definida en el Ejercicio 1.a) del Práctico 1. Hay que crear los usuarios `vendedor1`, `vendedor2` y `administrador`, otorgar los privilegios pedidos en cada inciso, verificar que quedaron bien aplicados y, al final, revocar todo lo que se le otorgó a `vendedor2`. Las decisiones de seguridad más relevantes son dos: usar `WITH GRANT OPTION` solo en `administrador` (porque el enunciado lo pide) y aplicar permisos a nivel de columna a `vendedor2` y `administrador` para acotar el alcance al mínimo necesario.

2.1. Conexión al servidor

Las operaciones de creación de usuarios y de `GRANT` requieren un usuario con privilegios de administración (típicamente `root`). El acceso desde la terminal se realiza así:

```
mysql -u root -p
```

Si conviene fijar host y puerto explícitamente, se puede usar:

```
mysql -h localhost -P 3306 -u root -p
```

Una vez dentro, vale la pena confirmar contra qué base se está trabajando:

```
SHOW DATABASES;
USE practico1a_mysql;
SELECT DATABASE(), CURRENT_USER();
```

2.2. Creación de usuarios

Los tres usuarios se crean con alcance `'@localhost'`, lo que tiene sentido en un escenario administrado desde la misma máquina. Las contraseñas se eligen suficientemente fuertes para cumplir con la política por defecto de MySQL 8 (`validate_password`).

```
CREATE USER IF NOT EXISTS 'vendedor1'@'localhost' IDENTIFIED BY 'Vendedor1_2026!';
CREATE USER IF NOT EXISTS 'vendedor2'@'localhost' IDENTIFIED BY 'Vendedor2_2026!';
```

```
CREATE USER IF NOT EXISTS 'administrador'@'localhost' IDENTIFIED BY 'Admin_2026!';
```

2.3. Otorgamiento de privilegios

Cada GRANT traduce literalmente uno de los incisos del enunciado y se acota al mínimo necesario.

```
-- a) vendedor1: puede INSERT en cliente, sin posibilidad de delegar.
-- Se omite WITH GRANT OPTION justamente porque el enunciado lo prohíbe.
GRANT INSERT ON practico1a_mysql.cliente TO 'vendedor1'@'localhost';

-- b) vendedor2: INSERT en factura y SELECT de los campos de producto.
-- Se otorga SELECT a nivel de columna en lugar de SELECT sobre toda la tabla,
-- para reflejar exactamente el alcance pedido.
GRANT INSERT ON practico1a_mysql.factura TO 'vendedor2'@'localhost';
GRANT SELECT (cod_producto, descripcion, precio, stock_minimo, stock_maximo, cantidad)
ON practico1a_mysql.producto
TO 'vendedor2'@'localhost';

-- c) administrador: UPDATE solo sobre el campo descripcion en producto.
-- Aquí sí corresponde WITH GRANT OPTION, porque el enunciado pide
-- que pueda delegar ese privilegio a otros usuarios.
GRANT UPDATE (descripcion)
ON practico1a_mysql.producto
TO 'administrador'@'localhost'
WITH GRANT OPTION;

-- d) DELETE sobre cliente para los tres usuarios. Se otorga en una sola
-- sentencia listando los tres beneficiarios separados por coma.
GRANT DELETE ON practico1a_mysql.cliente
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost', 'administrador'@'localhost';
```

2.4. Verificación de privilegios

SHOW GRANTS lista los permisos efectivamente registrados para cada cuenta y permite chequear de un vistazo que los GRANT se aplicaron como se esperaba.

```
SHOW GRANTS FOR 'vendedor1'@'localhost';
SHOW GRANTS FOR 'vendedor2'@'localhost';
SHOW GRANTS FOR 'administrador'@'localhost';
```

Más allá de la verificación declarativa, conviene probar el comportamiento real ingresando con cada usuario:

- **vendedor1**: el INSERT sobre `cliente` y el DELETE sobre `cliente` deberían funcionar; cualquier intento de `GRANT INSERT ON practico1a_mysql.cliente TO ...` debe fallar, justamente porque no se otorgó GRANT OPTION.
- **vendedor2**: el INSERT sobre `factura`, el SELECT sobre los campos permitidos de `producto` y el DELETE sobre `cliente` deberían ejecutarse sin problemas.
- **administrador**: el UPDATE del campo `descripcion` en `producto` debe funcionar y, como tiene WITH GRANT OPTION, también puede correr un `GRANT UPDATE (descripcion) ON ... TO ...` hacia otro usuario.

2.5. Revocación de privilegios de vendedor2

El último inciso pide retirar todos los permisos de **vendedor2**. La forma más directa en MySQL es combinar ALL PRIVILEGES con GRANT OPTION en un único REVOKE.

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'vendedor2'@'localhost';  
SHOW GRANTS FOR 'vendedor2'@'localhost';
```

Tras el REVOKE, el SHOW GRANTS debería devolver únicamente USAGE. Conviene aclarar que USAGE no es un privilegio sobre objetos: indica simplemente que la cuenta sigue existiendo y puede conectarse, pero ya no puede operar sobre ninguna tabla.

3. Ejercicio 2: Conexión por host y límites de recursos

Este ejercicio se resuelve sobre la base `practico1b_mysql`, definida en el inciso b) del Práctico 1. Lo característico de este caso, además de los privilegios sobre tablas, es que el enunciado pide controlar desde dónde puede conectarse cada usuario y limitar el consumo de recursos del cobrador.

El plan es crear `encargado_estacionamiento` con acceso restringido al servidor local, dar de alta a `cobrador` con conexión desde cualquier host, otorgar a cada uno los privilegios pedidos, configurar los límites de conexiones y consultas por hora del `cobrador`, verificar que todo quedó bien aplicado y, finalmente, revocar el permiso de consulta del `cobrador`.

3.1. Creación de usuarios con alcance de host

La parte del nombre que sigue al @ define el host desde el cual el usuario puede iniciar sesión. Eso es lo que permite restringir la conexión del `encargado_estacionamiento` al servidor local y dejar al `cobrador` con acceso desde cualquier máquina mediante el comodín '%'.

```
-- Limpieza opcional para re-ejecutar el script de manera idempotente  
DROP USER IF EXISTS 'encargado_estacionamiento'@'localhost';  
DROP USER IF EXISTS 'cobrador'@'%';  
  
-- encargado_estacionamiento: solo se conecta desde localhost  
CREATE USER 'encargado_estacionamiento'@'localhost'  
IDENTIFIED BY 'Encargado_2026!';  
  
-- cobrador: puede conectarse desde cualquier máquina  
CREATE USER 'cobrador'@'%'  
IDENTIFIED BY 'Cobrador_2026!';
```

3.2. Otorgamiento de privilegios y límites

Cada GRANT traduce un inciso del enunciado, y los límites del `cobrador` se configuran con ALTER USER ... WITH, que es la forma idiomática en MySQL 8 para fijar cuotas de uso por hora.

```
-- a) cobrador: solo SELECT sobre estacionamiento  
GRANT SELECT ON practico1b_mysql.estacionamiento  
TO 'cobrador'@'%';  
  
-- Límites de recursos: 3 conexiones/hora y 10 consultas/hora.  
-- Se aplican a la cuenta, no al privilegio en particular.  
ALTER USER 'cobrador'@'%'  
WITH  
    MAX_CONNECTIONS_PER_HOUR 3  
    MAX_QUERIES_PER_HOUR 10;  
  
-- b) encargado_estacionamiento: SELECT en todas las tablas e INSERT en parquimetro.  
-- El primer GRANT usa "*" para abarcar el esquema completo, y el segundo  
-- se acota a una sola tabla.
```

```
GRANT SELECT ON practico1b_mysql.*
TO 'encargado_estacionamiento'@'localhost';

GRANT INSERT ON practico1b_mysql.parquimetro
TO 'encargado_estacionamiento'@'localhost';
```

3.3. Verificación de permisos y de la configuración de cuenta

La verificación se realiza en dos planos: por un lado los privilegios efectivos sobre objetos, y por otro la configuración de la cuenta (host y límites de recursos).

```
-- Permisos efectivos sobre objetos
SHOW GRANTS FOR 'cobrador'@'%';
SHOW GRANTS FOR 'encargado_estacionamiento'@'localhost';

-- Definición de la cuenta: host de origen y límites declarados
SHOW CREATE USER 'cobrador'@'%';
SHOW CREATE USER 'encargado_estacionamiento'@'localhost';
```

La verificación se considera correcta cuando se observa lo siguiente:

- `cobrador@' %'` aparece con `SELECT` sobre `practico1b_mysql.estacionamiento`.
- `cobrador@' %'` muestra `MAX_CONNECTIONS_PER_HOUR 3` y `MAX_QUERIES_PER_HOUR 10` en `SHOW CREATE USER`.
- `encargado_estacionamiento@'localhost'` figura con `SELECT` sobre `practico1b_mysql.*` y con `INSERT` sobre `parquimetro`.

3.4. Revocación del privilegio de consulta del cobrador

```
REVOKE SELECT ON practico1b_mysql.estacionamiento
FROM 'cobrador'@'%';

SHOW GRANTS FOR 'cobrador'@'%';
```

Tras el `REVOKE`, la cuenta del `cobrador` y sus límites de recursos siguen vigentes, pero ya no tiene acceso de lectura a `estacionamiento`. El `SHOW GRANTS` debería devolver únicamente `USAGE`, lo que confirma que el único permiso pedido en el inciso a) fue retirado correctamente.

4. Ejercicio 3: Roles y herencia en PostgreSQL

Este ejercicio se trabaja en PostgreSQL sobre el esquema `practico1c` definido en el inciso c) del Práctico 1. A diferencia del primer ejercicio, acá entra en juego el modelo de roles de PostgreSQL: en este motor no existe una distinción real entre “usuario” y “rol”, ya que ambos son entidades del sistema y la única diferencia operativa es si el rol puede iniciar sesión (`LOGIN`) o no (`NOLOGIN`).

Esa característica permite resolver con naturalidad el pedido del enunciado: se crea un rol común `empleado` sin posibilidad de login, se le otorgan los permisos compartidos por todos los usuarios y luego se hace que `asesor`, `administrativo` y `encargado` (que sí tienen `LOGIN`) hereden de él.

4.1. Pre-chequeo del esquema

Antes de otorgar privilegios conviene confirmar que el esquema y las tablas existen:

```
SELECT schema_name
FROM information_schema.schemata
```

```
WHERE schema_name = 'practico1c';

SELECT tablename
FROM pg_tables
WHERE schemaname = 'practico1c'
ORDER BY tablename;
```

4.2. Creación del rol común y de los usuarios

El rol empleado se declara como NOLOGIN: no se utiliza para iniciar sesión, sino como contenedor de privilegios compartidos. Las tres cuentas que sí pueden conectarse se crean con LOGIN y, mediante GRANT empleado TO ..., heredan automáticamente lo que se otorgue al rol común.

```
-- Limpieza opcional para re-ejecutar el script de manera segura
DROP ROLE IF EXISTS asesor;
DROP ROLE IF EXISTS administrativo;
DROP ROLE IF EXISTS encargado;
DROP ROLE IF EXISTS empleado;

-- Rol común sin posibilidad de iniciar sesión
CREATE ROLE empleado NOLOGIN;

-- Usuarios reales: roles con LOGIN y contraseña
CREATE ROLE asesor LOGIN PASSWORD 'Asesor_2026!';
CREATE ROLE administrativo LOGIN PASSWORD 'Administrativo_2026!';
CREATE ROLE encargado LOGIN PASSWORD 'Encargado_2026!';

-- Inciso a): todos los usuarios pertenecen al rol empleado
GRANT empleado TO asesor, administrativo, encargado;
```

4.3. Otorgamiento de privilegios

Los privilegios se otorgan en el orden lógico del enunciado. Lo común a todos se asigna al rol empleado; lo específico se otorga directamente a cada usuario, que igual mantiene la herencia del rol.

```
-- a) empleado: SELECT a nivel de columnas (apellido, nombre) en cliente.
GRANT SELECT (apellido, nombre)
ON practico1c.cliente
TO empleado;

-- b) asesor: además de empleado, puede consultar taller e insertar en accidente.
GRANT SELECT ON practico1c.taller TO asesor;
GRANT INSERT ON practico1c.accidente TO asesor;

-- c) administrativo: SELECT sobre la tabla automovil.
GRANT SELECT ON practico1c.automovil TO administrativo;

-- d) encargado: DELETE sobre todas las tablas y UPDATE solo de la columna tasa
-- en categoria. ALL TABLES IN SCHEMA es un atajo cómodo para no listar
-- tabla por tabla.
GRANT DELETE ON ALL TABLES IN SCHEMA practico1c TO encargado;
GRANT UPDATE (tasa) ON practico1c.categoria TO encargado;
```

4.4. Comprobación de permisos

PostgreSQL no tiene un único SHOW GRANTS como MySQL, así que la verificación se arma combinando varias consultas al diccionario y funciones del estilo has_table_privilege y has_column_privilege.

La idea es chequear tres cosas: que los roles existen y pueden iniciar sesión cuando corresponde, que la membresía al rol `empleado` quedó bien establecida, y que cada privilegio puntual está efectivamente otorgado.

```
-- roles y capacidad de login
SELECT rolname, rolcanlogin
FROM pg_roles
WHERE rolname IN ('empleado', 'asesor', 'administrativo', 'encargado')
ORDER BY rolname;

-- membresía al rol empleado
SELECT r.rolname AS role, m.rolname AS member
FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.roleid
JOIN pg_roles m ON m.oid = am.member
WHERE r.rolname = 'empleado'
ORDER BY m.rolname;

-- permisos por tabla
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE table_schema = 'practico1c'
AND grantee IN ('empleado', 'asesor', 'administrativo', 'encargado')
ORDER BY grantee, table_name, privilege_type;

-- permisos por columna
SELECT grantee, table_name, column_name, privilege_type
FROM information_schema.column_privileges
WHERE table_schema = 'practico1c'
AND grantee IN ('empleado', 'encargado')
AND (
    (table_name = 'cliente' AND column_name IN ('apellido', 'nombre'))
    OR
    (table_name = 'categoria' AND column_name = 'tasa')
)
ORDER BY grantee, table_name, column_name;

-- chequeo puntual con funciones de privilegios
SELECT
    has_table_privilege('asesor', 'practico1c.taller', 'SELECT') AS
        asesor_select_taller,
    has_table_privilege('asesor', 'practico1c.accidente', 'INSERT') AS
        asesor_insert_accidente,
    has_table_privilege('administrativo', 'practico1c.automovil', 'SELECT') AS
        administrativo_select_automovil,
    has_table_privilege('encargado', 'practico1c.cliente', 'DELETE') AS
        encargado_delete_cliente,
    has_column_privilege('encargado', 'practico1c.categoria', 'tasa', 'UPDATE') AS
        encargado_update_tasa,
    has_column_privilege('asesor', 'practico1c.cliente', 'apellido', 'SELECT') AS
        asesor_select_apellido_via_empleado,
    has_column_privilege('administrativo', 'practico1c.cliente', 'nombre', 'SELECT') AS
        administrativo_select_nombre_via_empleado;
```

4.5. Resultado esperado por inciso

- empleado: `SELECT(apellido, nombre)` sobre `practico1c.cliente`; todo lo demás se hereda hacia los tres usuarios.
- asesor: lo heredado de empleado más `SELECT` sobre `practico1c.taller` e `INSERT` sobre `practico1c.accidente`.

- **administrativo**: lo heredado de **empleado** más **SELECT** sobre `practico1c.automovil`.
- **encargado**: lo heredado de **empleado** más **DELETE** sobre todas las tablas del esquema y **UPDATE(tasa)** sobre `practico1c.categoria`.

5. Ejercicio 4: Perfiles, vistas y delegación en Oracle

Este ejercicio se resuelve sobre el esquema **PRACTICO1C** (el del inciso c) del Práctico 1) y trabaja específicamente con las particularidades del modelo de seguridad de Oracle. Hay que crear los usuarios **empleado1**, **empleado2** y **director**, dar de alta el rol **empleados** y asignárselo a los dos primeros, otorgar los privilegios sobre objetos pedidos en cada inciso, forzar a **empleado1** a renovar su contraseña en el primer inicio de sesión y limitar a **director** a un máximo de 20 minutos de conexión continua. Esto último se logra con un **PROFILE** de Oracle, mecanismo específico del motor para imponer restricciones sobre recursos y contraseñas.

5.1. Nota sobre Oracle y privilegios por columna

A diferencia de MySQL o PostgreSQL, Oracle no admite **GRANT SELECT(col1, col2, ...)** directamente sobre una tabla. Para cumplir el pedido del inciso de **empleado2** (consultar **apellido**, **nombre** y **direccion** de **cliente** y poder delegar ese privilegio), se crea una vista con exactamente esas columnas y se otorga **SELECT ... WITH GRANT OPTION** sobre la vista. Es la solución idiomática en Oracle: la vista actúa como una proyección controlada de la tabla y el **WITH GRANT OPTION** queda acotado a esa proyección.

5.2. Bloque de administración (conexión como SYS)

La parte de creación de usuarios, roles y perfiles se ejecuta con privilegios de administrador, así que se ingresa por **sqlplus** como **SYSDBA**. La instrucción **ALTER SYSTEM SET RESOURCE_LIMIT = TRUE** es necesaria para que Oracle aplique los límites declarados en los perfiles.

```
-- sqlplus / as sysdba

-- Habilitar la aplicación de límites de recursos definidos en los perfiles.
-- Sin esta línea, CONNECT_TIME no surte efecto sobre director.
ALTER SYSTEM SET RESOURCE_LIMIT = TRUE;

-- Creación de los tres usuarios. Se les asigna tablespace por defecto y temporal,
-- más una cuota mínima en USERS por si llegan a crear objetos propios.
CREATE USER empleado1 IDENTIFIED BY "Empleado1_2026!"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON users;

CREATE USER empleado2 IDENTIFIED BY "Empleado2_2026!"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON users;

CREATE USER director IDENTIFIED BY "Director_2026!"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON users;

-- Rol que agrupa los privilegios comunes a empleado1 y empleado2.
CREATE ROLE empleados;

-- Privilegio de sistema necesario para iniciar sesión: sin CREATE SESSION
-- la cuenta existe pero no puede conectarse al motor.
GRANT CREATE SESSION TO empleado1, empleado2, director;
```

```

-- Asignación del rol a los dos empleados (director queda fuera).
GRANT empleados TO empleado1, empleado2;

-- Para forzar a empleado1 a renovar su contraseña en el primer login,
-- se marca la actual como expirada.
ALTER USER empleado1 PASSWORD EXPIRE;

-- Perfil que limita a 20 minutos el tiempo total de conexión continua.
-- En Oracle, CONNECT_TIME se mide en minutos.
CREATE PROFILE director_profile LIMIT
  CONNECT_TIME 20;

ALTER USER director PROFILE director_profile;

```

5.3. Bloque de privilegios sobre objetos (conexión como PRACTICO1C)

Los GRANT sobre tablas y vistas deben ejecutarse desde el dueño del esquema, ya que es quien tiene autoridad para entregar privilegios sobre sus propios objetos. Por eso se cambia de sesión al usuario PRACTICO1C.

```

-- sqlplus practico1c/practico1c

-- El rol empleados consulta AUTOMOVIL y CATEGORIA. Como el rol ya está asignado
-- a empleado1 y empleado2, el privilegio se hereda automáticamente a ambos.
GRANT SELECT ON automovil TO empleados;
GRANT SELECT ON categoria TO empleados;

-- director debe poder consultar todas las tablas del esquema. Para no listarlas
-- a mano, se itera sobre user_tables y se otorga SELECT dinámicamente.
BEGIN
  FOR t IN (SELECT table_name FROM user_tables) LOOP
    EXECUTE IMMEDIATE 'GRANT SELECT ON ' || t.table_name || ' TO director';
  END LOOP;
END;
/

-- empleado1: privilegios DML completos sobre accidente.
GRANT INSERT, UPDATE, DELETE ON accidente TO empleado1;

-- empleado2: vista que expone exactamente apellido, nombre y direccion de cliente.
-- Después se otorga SELECT sobre la vista con WITH GRANT OPTION para que pueda
-- delegar el privilegio, tal como pide el enunciado.
CREATE OR REPLACE VIEW vw_cliente_datos AS
SELECT apellido, nombre, direccion
FROM cliente;

GRANT SELECT ON vw_cliente_datos TO empleado2 WITH GRANT OPTION;

```

5.4. Verificación

La verificación se hace combinando varias vistas del diccionario de Oracle. Cada una mira un aspecto puntual: existencia de usuarios y rol, membresía al rol, privilegios directos sobre objetos, capacidad de delegar y configuración del perfil de director.

```

-- Usuarios creados
SELECT username, account_status, profile
FROM dba_users
WHERE username IN ('EMPLEADO1', 'EMPLEADO2', 'DIRECTOR')
ORDER BY username;

```

```
-- Rol empleados creado
SELECT role
FROM dba_roles
WHERE role = 'EMPLEADOS';

-- Membresía del rol
SELECT grantee, granted_role
FROM dba_role_privs
WHERE grantee IN ('EMPLEADO1', 'EMPLEADO2', 'DIRECTOR')
ORDER BY grantee, granted_role;

-- Privilegios del rol empleados
SELECT grantee, owner, table_name, privilege
FROM dba_tab_privs
WHERE grantee = 'EMPLEADOS'
  AND owner = 'PRACTICO1C'
ORDER BY table_name, privilege;

-- Privilegios directos de empleado1 y director
SELECT grantee, owner, table_name, privilege
FROM dba_tab_privs
WHERE grantee IN ('EMPLEADO1', 'DIRECTOR')
  AND owner = 'PRACTICO1C'
ORDER BY grantee, table_name, privilege;

-- Vista para empleado2 y capacidad de delegar
SELECT grantee, owner, table_name, privilege, grantable
FROM dba_tab_privs
WHERE grantee = 'EMPLEADO2'
  AND owner = 'PRACTICO1C'
  AND table_name = 'VW_CLIENTE_DATOS';

-- Configuración del perfil de director
SELECT profile, resource_name, limit
FROM dba_profiles
WHERE profile = 'DIRECTOR_PROFILE'
  AND resource_name = 'CONNECT_TIME';
```

5.5. Resultado esperado por inciso

A modo de resumen, después de ejecutar todos los bloques anteriores se debería observar lo siguiente:

- empleado1 y empleado2 aparecen con el rol empleados asignado.
- El rol empleados tiene SELECT sobre AUTOMOVIL y CATEGORIA, privilegios que se heredan a ambos usuarios.
- empleado1 cuenta con INSERT, UPDATE y DELETE directos sobre ACCIDENTE, y su contraseña queda como EXPIRED, forzando el cambio en el primer ingreso.
- empleado2 tiene SELECT sobre VW_CLIENTE_DATOS (apellido, nombre, dirección) con GRANTABLE = YES, lo que confirma la posibilidad de delegar.
- director queda con SELECT sobre todas las tablas del esquema PRACTICO1C y con el perfil DIRECTOR_PROFILE, donde CONNECT_TIME = 20.

6. Conclusión

El práctico permitió ejercitar el control de accesos en tres motores con modelos de seguridad bastante distintos. En MySQL, la identidad del usuario se forma con el par `usuario@host` y

los privilegios pueden afinarse a nivel de columna; los límites de recursos por hora se gestionan directamente sobre la cuenta con `ALTER USER ... WITH`. En PostgreSQL, “usuarios” y “roles” son la misma cosa, y la herencia entre roles permite agrupar permisos comunes en un único contenedor, lo que simplifica la administración. En Oracle, en cambio, hay que apoyarse en perfiles para limitar recursos como el tiempo de conexión y, ante la falta de `GRANT SELECT(col)` sobre tablas, recurrir a vistas para acotar la visibilidad de columnas.

Más allá de las diferencias entre motores, el ejercicio refuerza dos principios generales de seguridad: otorgar el privilegio mínimo necesario y delegar (con `WITH GRANT OPTION`) solo cuando hay una razón explícita para hacerlo. Las verificaciones con `SHOW GRANTS`, las vistas `information_schema` y `pg_*` en PostgreSQL y las vistas `DBA_*` en Oracle resultaron fundamentales para validar que los `GRANT` y `REVOKE` produjeron el efecto esperado.